ISEL
DEETC
Departamento de Engenharia Electrónica e de Telecomunicações e de Computadores

Aula 11

Sumário

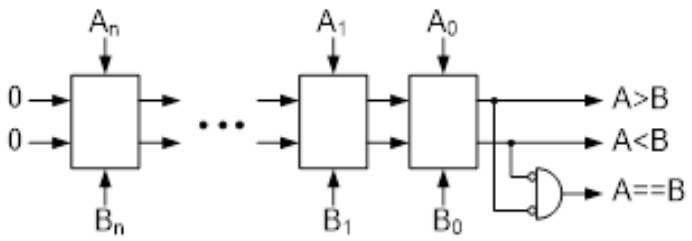
- Operadores relacionais
- *Arithmetic Logic Unit* (ALU)

Comparação de números

- A comparação entre dois números, A e B, é realizada para determinar se:
  - A e B são iguais
  - A é maior que B
  - A é menor que B

Operadores relacionais

- Um circuito comparador é capaz de determinar a relação entre dois números:
  - Igual
  - Maior
  - Menor
- Pode ser implementado recorrendo a uma solução iterativa:
  - uma subtracção com posterior análise do resultado
  - comparação dígito a dígito, começando do mais significativo para o menos significativo:



Operadores relacionais

- Usando um circuito somador/subtractor é possível obter a relação entre os operandos analisando o resultado da subtracção:
  - A - B > 0 => A > B
  - A - B < 0 => A < B
  - A - B = 0 => A = B
- Para operandos de *n* bits representando números naturais:
  - A = B se ($S_{n-1} = S_{n-2} = \ldots = S_1 = S_0 = 0$)
  - A > B se CyBwOut = 0 e A ≠ B
  - A < B se CyBwOut = 1
- Para operandos de *n* bits representando números inteiros (complemento para 2):
  - A = B se ($S_{n-1} = S_{n-2} = \ldots = S_1 = S_0 = 0$)
  - A > B se ($S_{n-1} = 0$ e OV = 0 e A ≠ B) ou ($S_{n-1} = 1$ e OV = 1)
  - A < B se ($S_{n-1} = 1$ e OV = 0) ou ($S_{n-1} = 0$ e OV = 1)

Operadores relacionais

- A relação entre os operandos é indicada através de *flags* que são válidas para operandos no correspondente domínio

	Menor (<)	Igual (=)	Maior (>)
Naturais ($\mathbb{N}$)	**Below (B)**	**Equal**	**Above (A)**
Inteiros ($\mathbb{Z}$)	**Less (L)**		**Greater (G)**

Operadores relacionais

- Para operandos de *n* bits representando números naturais:
  - $(A = B)$ Equal (Zero): $E = Z = \overline{S_{n-1} + \cdots + S_1 + S_0}$
  - $(A > B)$ Above: $A = \overline{CyBw_n}.\overline{Z}$
  - $(A < B)$ Below: $B = CyBw_n$

estudar estas exrpessões

- Para operandos de *n* bits representando números inteiros (complemento para 2):
  - $(A = B)$ Equal: $E = Z = \overline{S_{n-1} + \cdots + S_1 + S_0}$
  - $(A > B)$ Greater: $G = \overline{S_{n-1}}.\overline{OV}.\overline{Z} + S_{n-1}.OV$
  - $(A < B)$ Less: $L = S_{n-1}.\overline{OV} + \overline{S_{n-1}}.OV = S_{n-1} \oplus OV$

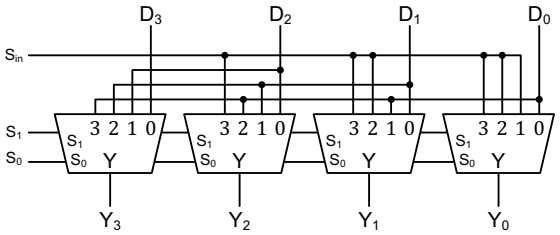
- As *flags* podem representar associações dos operadores relacionais:
  - Por exemplo, a *flag* GE (Greater or Equal), indica se $A \geq B$ para operandos no domínio dos inteiros

Operação shift

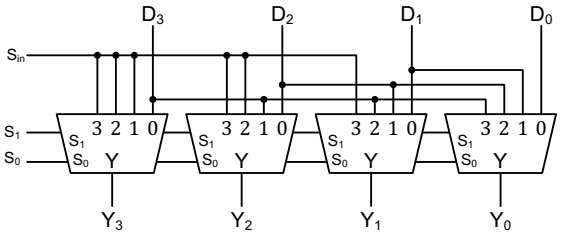
- Na maioria das linguagens de alto nível existe uma operação de *shitf* que permite manipular bit a bit um determinado operando (e.g. com 16 bits)
  - *shift-left and shift-right*
  - *arithmetic, logical and circular shifts (rotate)*
- Pode ser usado para multiplicar ou dividir por potências inteiras de 2
  - multiplicar por 2^n pode ser obtido por um *arithmetic shitf-left* de *n* posições
  - dividir por 2^m pode ser obtido por um *arithmetic shitf-right* de *m* posições
- Pode ser usado na manipulação de palavras de *n* bits para extrair partes específicas
  - extrair o valor codificado nos bits 3..10 de uma palavra com 16 bits
    - 0101101011001010

Barrel Shifter

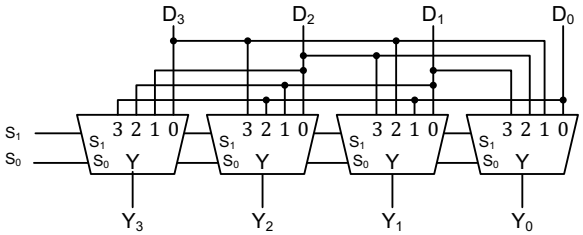
- Implementa a operação de deslocamento (shift) em n posições
- Entradas
  - $D_{3..0}$ – operando sobre o qual se vai efectuar o deslocamento
  - $S_{1..0}$ – valor do deslocamento a aplicar ao operando D
  - S_{in} – valor a introduzir nos bits criados (à direita ou à esquerda consoante se faça um *shift-left* ou *shift-right*)



Shift Left



Shift Rigth



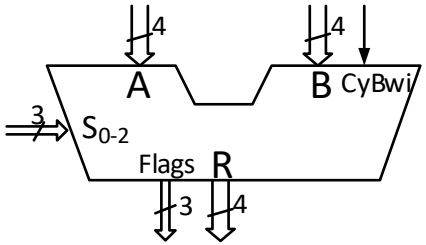
Rotate

Arithmetic Logic Unit - ALU

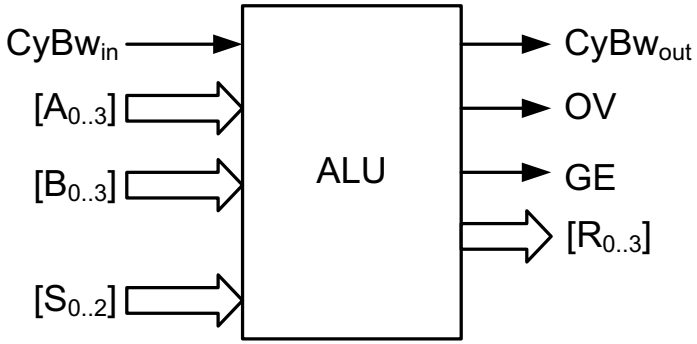
- A *Arithmetic Logic Unit* (ALU) implementa as funções lógicas e aritméticas sobre valores inteiros
- As ALUs não implementam todas o mesmo um conjunto de operações/flags.
- Algumas das operações típicas:
  - $R = A + B + CyBw_{in}$
  - $R = A - B - CyBw_{in}$
  - $R = A + 1$
  - $R = A - 1$
  - $R = A \,\&\, B$
  - $R = A \mid B$
  - $R = A \ll 1$

Arithmetic Logic Unit - ALU

- A *Arithmetic Logic Unit* (ALU) implementa as funções lógicas e aritméticas sobre valores inteiros
- As ALUs não implementam todas o mesmo um conjunto de operações/flags.
- Símbolos:

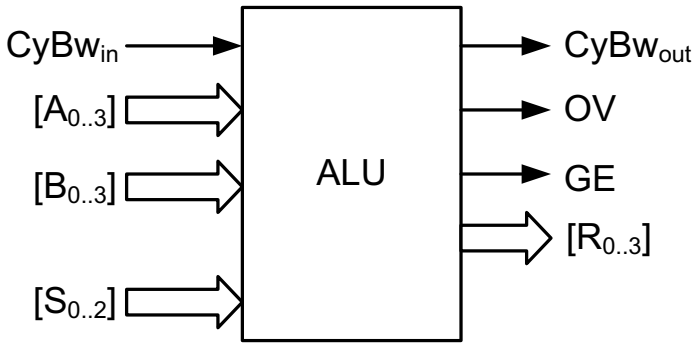


≡

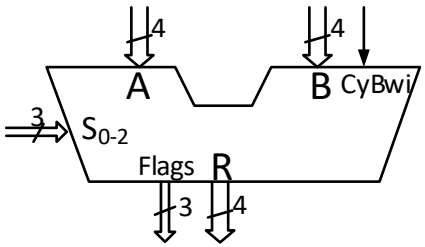


Arithmetic Logic Unit - ALU

- Algumas das operações mais comuns:
  - $R = A + B + CyBw_{in}$
  - $R = A - B - CyBw_{in}$
  - $R = A + 1$
  - $R = A - 1$
  - $R = A \,\&\, B$
  - $R = A \mid B$
  - $R = A \ll 1$



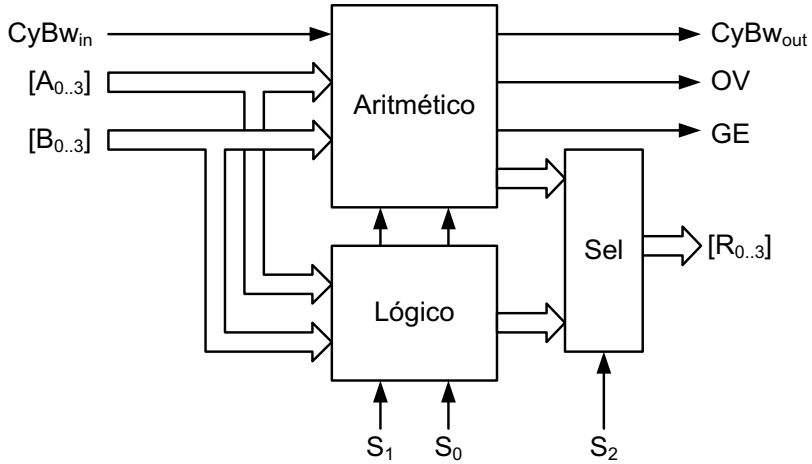
- Flags mais comuns:
  - CyBw (CarryBorrow)
  - OV (overflow)
  - GE (greater or equal)
  - Z (zero)



Arithmetic Logic Unit - ALU

- A operação a ser executada a cada momento é determinada através da entrada de seleção S
  - Exemplo de uma possível codificação:

S_2	S_1	S_0	Operação	$CyBw_{out}$	OV	GE
0	0	0	ADDC	•	•	-
0	0	1	SUBB	•	•	•
0	1	0	INC	•	•	-
0	1	1	DEC	•	•	-
1	0	0	AND	-	-	-
1	0	1	OR	-	-	-

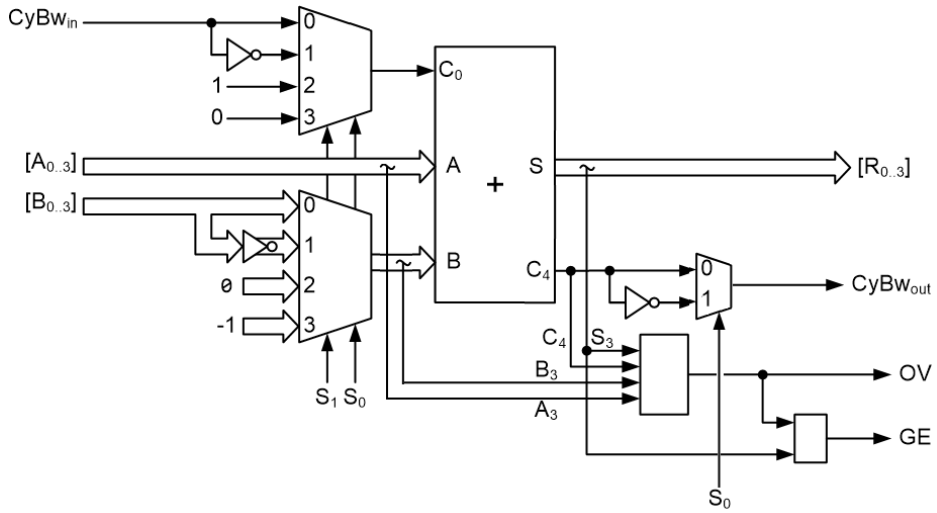


- Dois grandes blocos funcionais:
  - Aritmético
  - Lógico

Arithmetic Logic Unit - ALU

- Bloco aritmético
  - Baseia-se na utilização de um somador de *n* bits
  - As restantes operações são executadas recorrendo à manipulação das entradas deste somador

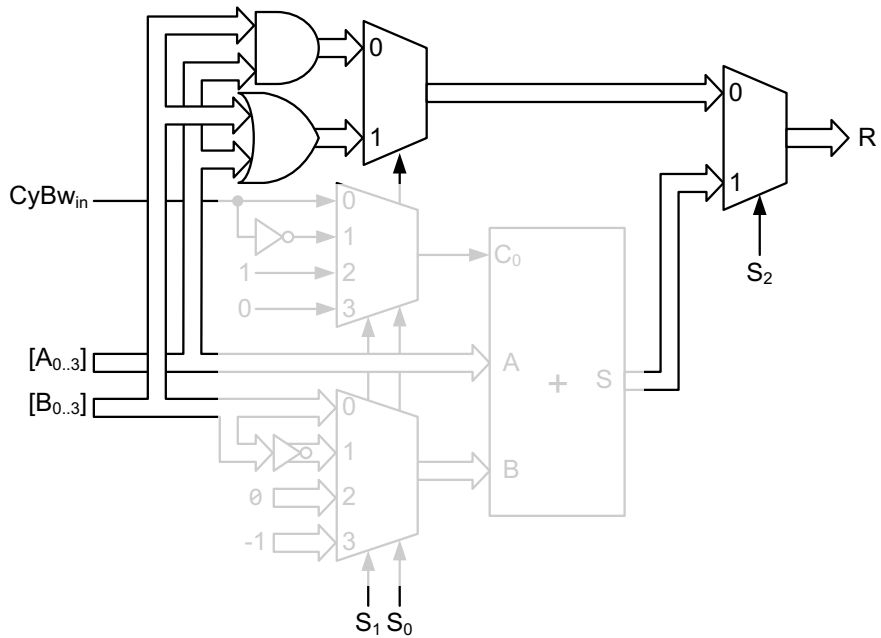
S_2	S_1	S_0	Operação	$CyBw_{out}$	OV	GE
0	0	0	ADDC	•	•	-
0	0	1	SUBB	•	•	•
0	1	0	INC	•	•	-
0	1	1	DEC	•	•	-



Arithmetic Logic Unit - ALU

- Bloco lógico
  - Implementa funções lógicas bit a bit (AND, OR)

S_2	S_1	S_0	Operação	$CyBw_{out}$	OV	GE
0	0	0	ADDC	•	•	-
0	0	1	SUBB	•	•	•
0	1	0	INC	•	•	-
0	1	1	DEC	•	•	-
1	0	0	AND	-	-	-
1	0	1	OR	-	-	-



Temas abordados

Sumário

- Operadores relacionais
- *Arithmetic Logic Unit* (ALU)

Próxima aula

- Laboratório C